Module: Strings

Phase: Core Java
Previous module: Flow Control
Next module: Arrays

Module Outcome:

- Slice and compare names, emails, and UPI IDs with `String` methods
- Format a receipt line instead of glueing commas by hand
- Remember `String` is immutable — `concat` without storing the result is a no-op

What is a string?

A **string** is a sequence of characters — names, emails, UPI IDs, error messages.



Where used?
- **SecureBank** / UPI — mask account/phone, parse UPI IDs
- College apps — roll numbers, email normalize for login
- **TechMart** — product titles, invoice lines
- **AcmeHR** — employee name / email formatting
- **MedCare** — phone masking, appointment subject cleanup

Why do we need strings?

Without string skills	With string skills
Messy `"  Ravi@Gmail.COM "` breaks login	`trim()` + `toLowerCase()` normalizes input
Show full phone on screen	Mask as `******3210`
`==` looks like it works in a demo	`equals` is the interview-safe compare

```
String name = "Ananya Sharma";
String course = "Java Full Stack";
```

Java has **`char`** for one character (`'A'`). A `String` of length 1 is still a `String`.

Immutability

Definition: After you create a `String`, you cannot change those characters in place.
Purpose: Safe sharing; methods like `toUpperCase()` return a **new** string.

```
String s = "priya";
s.toUpperCase();           // result ignored
```

```
System.out.println(s);     // still priya
s = s.toUpperCase();       // now PRIYA
```

Indexing

Indices start at **0**.

```
  P  r  i  y  a
  0  1  2  3  4
```

```
String lang = "Java";
System.out.println(lang.charAt(0));                // J
System.out.println(lang.substring(1, 3));          // av  (end exclusive)
```

Important Concepts and Definitions

Term	Definition
String	Immutable sequence of characters
Literal	Text in `"double quotes"`
Index	Zero-based position
substring(start, end)	Piece; `end` not included
equals	Content compare for objects
==	Same object (reference) — dangerous for input text
trim	Remove leading/trailing whitespace
StringBuilder	Mutable buffer for building text

Step-by-Step Notes (With Annotated Code)

Step 1: Create and print

```
public class StringCreate {
    public static void main(String[] args) {
        String a = "Priya";
        String b = new String("Priya");
        System.out.println(a);
        System.out.println(b);
        System.out.println(a.length());
    }
}
```

Expected Output:

```
Priya
Priya
5
```

Print the length of an empty string, a string of three spaces, and that same string after `trim`.

```
public class EmptyString {
    public static void main(String[] args) {
        String empty = "";
        String spaces = "   ";
        System.out.println(empty.length());
        System.out.println(spaces.length());
        System.out.println(spaces.trim().length());
    }
}
```

Expected Output:

```
0
3
0
```

Store first name Ravi and last name Kumar, then print them joined with a space.

```
public class ConcatName {
    public static void main(String[] args) {
        String first = "Ravi";
        String last = "Kumar";
        System.out.println(first + " " + last);
    }
}
```

Step 2: Immutability

```
public class ImmutableDemo {
    public static void main(String[] args) {
        String email = "  Ravi@Gmail.COM ";
        String cleaned = email.trim().toLowerCase();
        System.out.println("raw: [" + email + "]");
        System.out.println("clean: [" + cleaned + "]");
    }
}
```

Expected Output:

```
raw: [  Ravi@Gmail.COM ]
clean: [ravi@gmail.com]
```

Note:
- Wrong: Calling `trim()` and not storing the result.
- Prefer: `email = email.trim();` or a new variable.

Call `replace` on `dosa` without storing the result, then print the original string.

```
public class ReplaceIgnore {
    public static void main(String[] args) {
        String s = "dosa";
        s.replace("dosa", "idli");
        System.out.println(s);
    }
}
```

Expected Output:

```
dosa
```

Assign `s = s.replace("dosa", "idli")` and print the new string.

```
public class ReplaceKept {
    public static void main(String[] args) {
        String s = "dosa";
        s = s.replace("dosa", "idli");
        System.out.println(s);
    }
}
```

Expected Output:

```
idli
```

Step 3: Index and substring

```
public class SliceEmail {
    public static void main(String[] args) {
        String email = "priya@ksr.in";
        int at = email.indexOf('@');
        String user = email.substring(0, at);
        String domain = email.substring(at + 1);
        System.out.println(user);
        System.out.println(domain);
        System.out.println(email.charAt(0));
    }
}
```

Expected Output:

```
priya
ksr.in
p
```

Common Error: `charAt(5)` on `"Java"` → `StringIndexOutOfBoundsException` (length is 4).

Print the last character of `Pune` using `charAt` and `length() - 1`.

```
public class LastChar {
    public static void main(String[] args) {
        String city = "Pune";
        System.out.println(city.charAt(city.length() - 1));
    }
}
```

Print the first three characters of roll `2026001` with `substring`.

```
public class FirstThree {
    public static void main(String[] args) {
        String roll = "2026001";
        System.out.println(roll.substring(0, 3));
    }
}
```

Expected Output:

```
202
```

Step 4: equals vs ==

```
import java.util.Scanner;

public class EqualsVsEq {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("PIN: ");
        String pin = sc.nextLine();
        System.out.println(pin == "1234");
        System.out.println(pin.equals("1234"));
        sc.close();
    }
}
```

If the user types `1234`, `==` is often **false** (different objects). `equals` is **true**.

Interview: Always `equals` for `String` content. `==` is reference identity.

Compare stored name `Priya` with typed `priya` using `equals` and `equalsIgnoreCase`, then print both results.

```
public class IgnoreCase {
    public static void main(String[] args) {
        String stored = "Priya";
        String typed = "priya";
        System.out.println(stored.equals(typed));
        System.out.println(stored.equalsIgnoreCase(typed));
    }
}
```

Expected Output:

```
false
true
```

Check UPI ID `ravi@okaxis` with `contains("@")` and `startsWith("ravi")`, then print both booleans.

```
public class ContainsDemo {
    public static void main(String[] args) {
        String upi = "ravi@okaxis";
        System.out.println(upi.contains("@"));
        System.out.println(upi.startsWith("ravi"));
    }
}
```

Step 5: Clean email and mask phone

```
public class CleanMask {
    public static void main(String[] args) {
        String emailRaw = "  Ravi@Gmail.COM ";
        String email = emailRaw.trim().toLowerCase();

        String phoneRaw = "9876543210";
        String mask = "******" + phoneRaw.substring(phoneRaw.length() - 4);

        System.out.println(email);
        System.out.println(mask);
    }
}
```

Expected Output:

```
ravi@gmail.com
******3210
```

Digits-only phone (loop over characters):

```
public class DigitsOnly {
    public static void main(String[] args) {
        String raw = "987-654-3210";
        String digits = "";
        for (int i = 0; i < raw.length(); i++) {
            char c = raw.charAt(i);
            if (c >= '0' && c <= '9') {
                digits = digits + c;
            }
        }
        System.out.println(digits);
        System.out.println(digits.length() == 10);
    }
}
```

Lab kit: 15_Lab_Assets/06_string_parser.

Trim and lower-case `Hello`, then print it inside square brackets.

```
public class Normalize {
    public static void main(String[] args) {
        String s = "  Hello ";
        System.out.println("[" + s.trim().toLowerCase() + "]");
    }
}
```

Mask a SecureBank account number `123456789012` so only the last four digits show after asterisks.

```
public class MaskFour {
    public static void main(String[] args) {
        String acc = "123456789012";
        System.out.println("********" + acc.substring(acc.length() - 4));
    }
}
```

Step 6: StringBuilder lite

```
public class BuilderLite {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        sb.append("Priya");
        sb.append(" ");
        sb.append("Nair");
        System.out.println(sb.toString());
    }
}
```

Expected Output:

```
Priya Nair
```

Note: For a few `+` joins, `+` is fine. For building in a **long loop**, `StringBuilder` is the interview answer.

Practice Exercises

Basic

1. Print length of `"Hyderabad"`.

Model answer: `System.out.println("Hyderabad".length());` → `9`

2. `substring` of `"Hello"` from 1 to 4.

Model answer: `"ell"` because end is exclusive.

3. `trim` `" Hi "`.

Model answer: `"Hi"`

4. Why is `==` risky for `Scanner` input vs `"1234"`?

Model answer: `==` checks the same object. Typed input is a new `String`. Use `equals`.

Intermediate

1. Normalize email: trim + lower.

Model answer: `email.trim().toLowerCase()`

2. Mask a 10-digit phone as `******3210`.

Model answer: `"******" + phone.substring(6)` if length is 10.

3. Extract domain after `@`.

Model answer: `email.substring(email.indexOf('@') + 1)`

4. Digits-only from `"987-65-43210"` and print whether length is 10.

Model answer: Loop `charAt`; keep `'0'`–`'9'`; compare `length() == 10`.

Interview-Oriented

1. Are strings mutable in Java?

Model answer: No. Methods return new strings. `StringBuilder` is mutable.

2. `equals` vs `==`?

A: `equals` → content. `==` → same reference (except interned literals may look equal — do not rely on that in apps).

3. Why `indexOf` returns `-1`?

A: The piece was not found. Always check before `substring`.

4. `charAt` vs `substring` for one character?

A: `charAt` returns `char`. `substring(i, i+1)` returns a `String` of length 1.

Mini-Project (Assignment): AcmeHR / Admissions Field Cleaner

Lab kit: 15_Lab_Assets/06_string_parser

What to build

Clean messy email (trim + lower). Keep digits from a phone; require length 10; mask as `******XXXX`.

Stretch

- Reject empty email after trim (print a message, no `throw`).
- **Later — Methods:** put cleaners in static methods.

Done when

- [ ] Email normalized
- [ ] Phone digits-only + length 10 check
- [ ] Mask shows last 4 only
- [ ] `equals` used for any PIN/text compare in extras

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

Q1. What does immutable mean for String?

A: The character data cannot change. “Updates” create a new object.

Q2. Why not `==` for login password strings?

A: User input is a new object. `==` can be false even when letters match. Use `equals`.

Q3. What is `substring(1, 4)` on `"Hello"`?

A: `"ell"` — start inclusive, end exclusive.

Q4. How do you strip spaces from both ends?

A: `trim()` (or `strip()` on newer JDKs — `trim` is enough here).

Q5. When mention StringBuilder in an interview?

A: Building large text in a loop. Small concatenations of a few pieces can stay `+`.